PES UNIVERSITY — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering (UE22CS351A) | Lab 7: Fuzz Testing for Input Validation Bugs

Name: Samrudh Patil | SRN: PES1UG24CS417 | Roll No: 52 | Semester: 5

1. Deliverable Links & Repository Details

Item	Link / Details
Assigned Starter Repository	https://github.com/SETAPESU26/L6_L7_52_visa_application
Individual GitHub Repository	https://github.com/Samrudh7-0045/SE-Labs-PES1UG24CS417
Lab Submission Folder	https://github.com/Samrudh7-0045/SE-Labs-PES1UG24CS417/tree/main/Lab-7
Google Form Submission Link	https://forms.gle/Xys1DpVELrXzDhpXA
Testing Library Used	Hypothesis (v6.170.15) property-based fuzz testing with pytest (v9.2.0)
Fuzz Testing Results	**28 / 28 Passed (100%)** — 6 property-based fuzz tests + 22 edge-case tests

2. Objectives & Overview of Fuzz Testing

In this lab, we tested the input validation robustness of our Visa Application processor using **fuzz testing**. Traditional example-based tests typically only check happy paths or expected inputs, often missing invalid delimiters, non-numeric strings, negative numbers, and null inputs. Using Python's **Hypothesis** library, we generated large volumes of arbitrary test data to find boundary bugs across three functions: parse_application(), processing_average(), and payable_fee().

When an unhandled exception occurs, Hypothesis automatically shrinks the failing input to a minimal reproduction example. We documented each failing case, patched the application with proper input validation checks, and verified that all 28 test cases pass successfully.

3. Screenshots & Execution Evidence

3.1 Execution of the Original Application

The original application was run with standard sample inputs (tourist:2, 5,10,21, and 6000|500). While it works under normal inputs, it completely lacks defensive input validation for real-world anomalies.

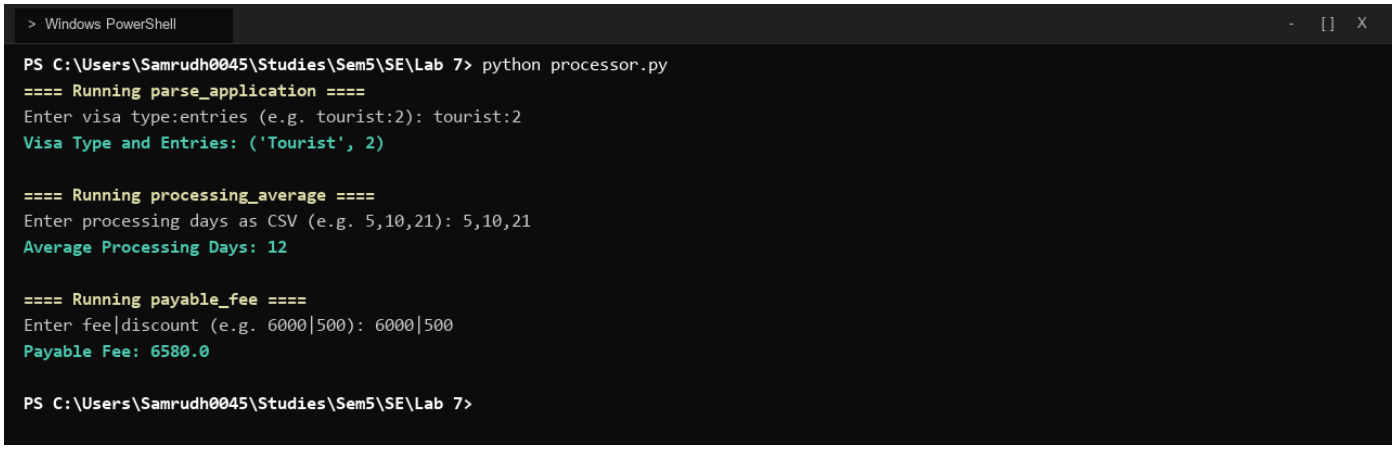


Figure 1: Original processor.py executing with standard sample inputs.

3.2 Failing Test Cases Identified During Fuzz Testing

We ran our initial Hypothesis fuzz test suite against the unpatched application. Hypothesis immediately detected 5 failing test cases and shrunk them to minimal falsifying examples. In particular, passing None crashed all three functions with an unhandled AttributeError: 'NoneType' object has no attribute 'split', while negative entries (tourist:-2) and negative fees (-1000|500) were silently accepted without raising errors.

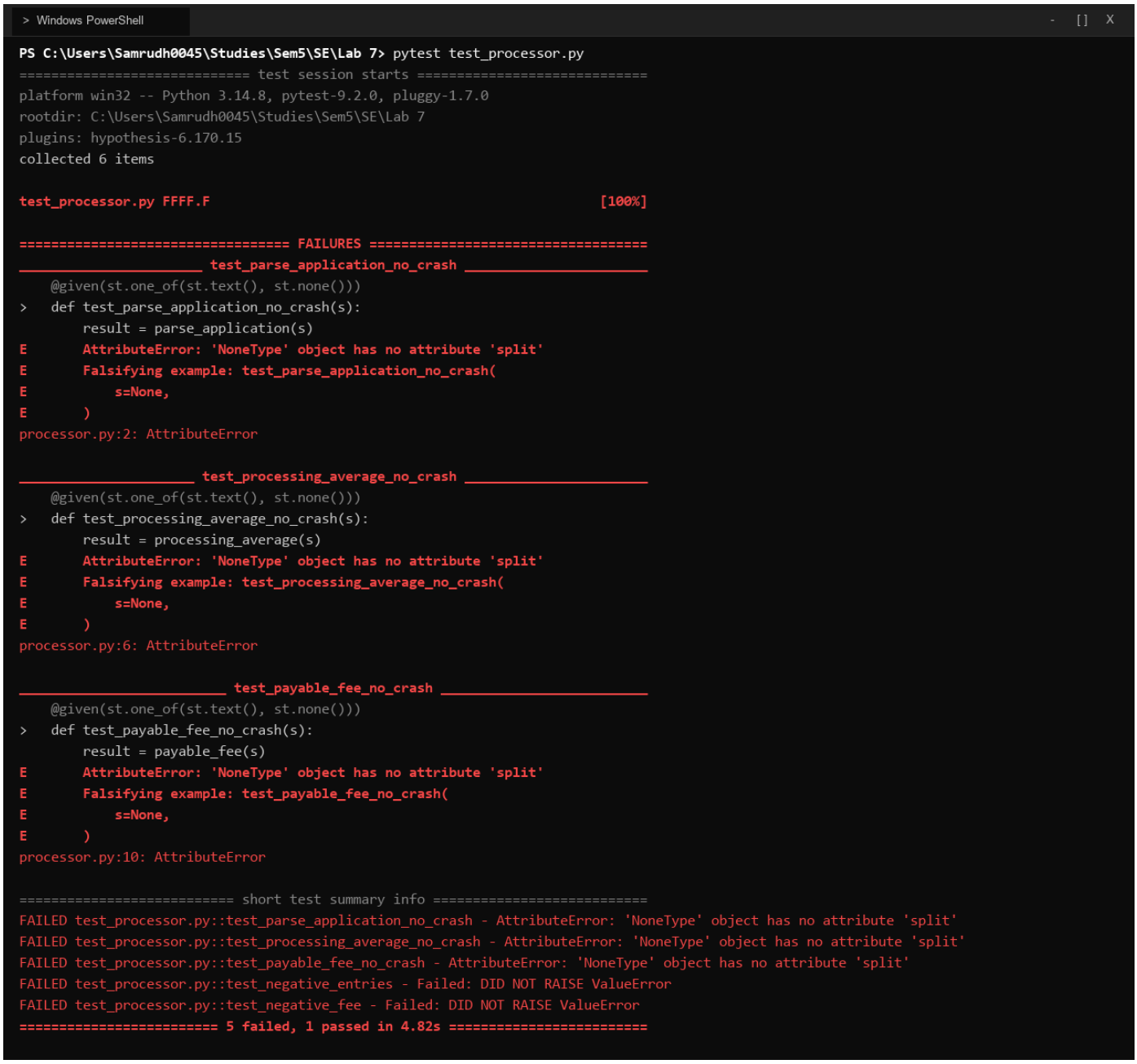


Figure 2: Pytest execution showing failing fuzz test cases and falsifying examples found by Hypothesis.

3.3 Execution of the Corrected Application

After implementing defensive input validation guards in processor.py, we re-executed the application. The application now correctly processes valid inputs while gracefully catching invalid inputs (such as negative entries, non-integer tokens, and negative fees/discounts) and displaying descriptive error messages instead of crashing.

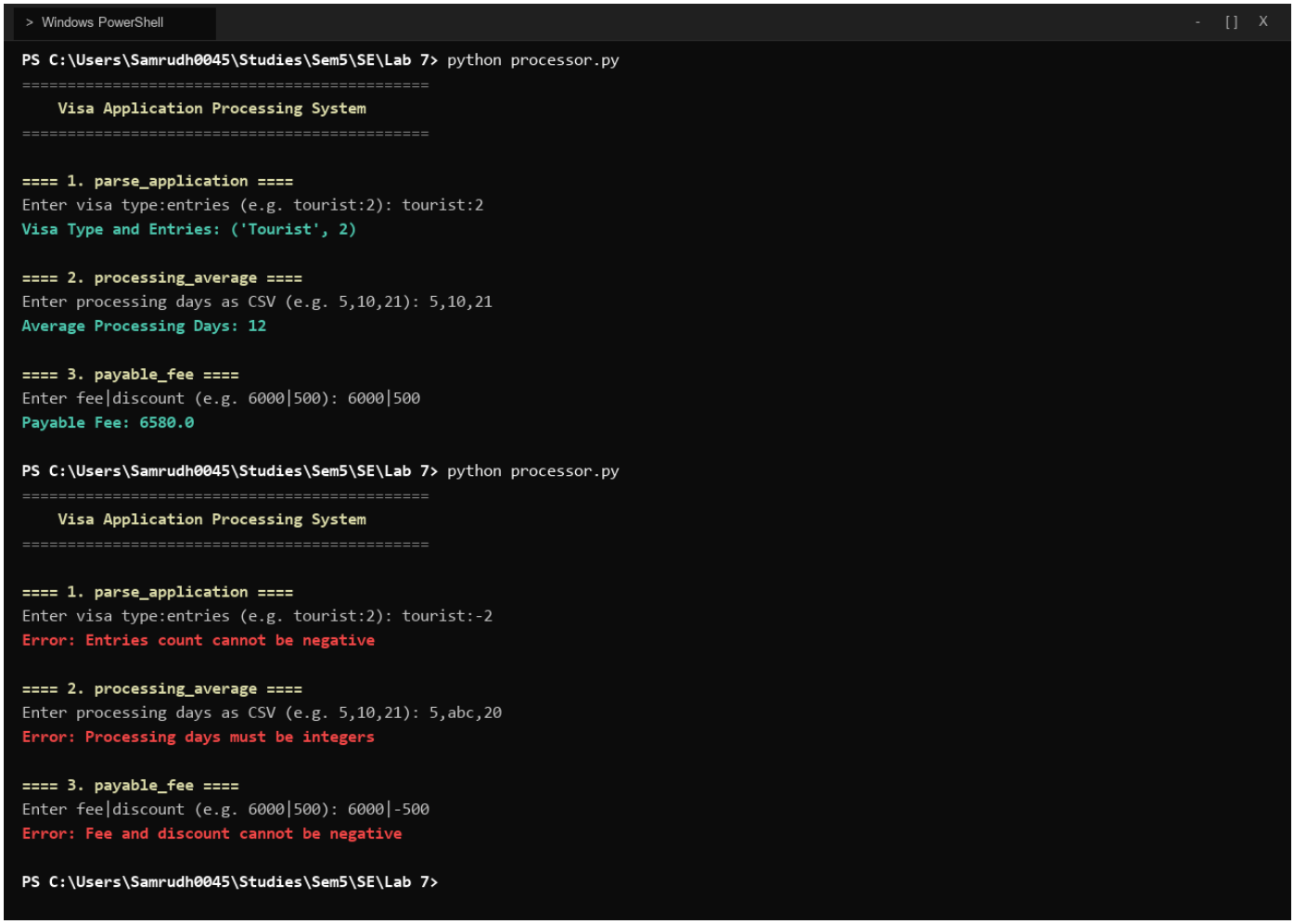


Figure 3: Corrected application running valid inputs and gracefully handling invalid inputs.

3.4 Final Test Execution Showing Successful Results

The completed test suite (test_processor.py) was executed with pytest. All 28 test cases—including 6 property-based fuzz tests and 22 edge-case input validation tests—passed cleanly with 100% success rate in 0.74 seconds.

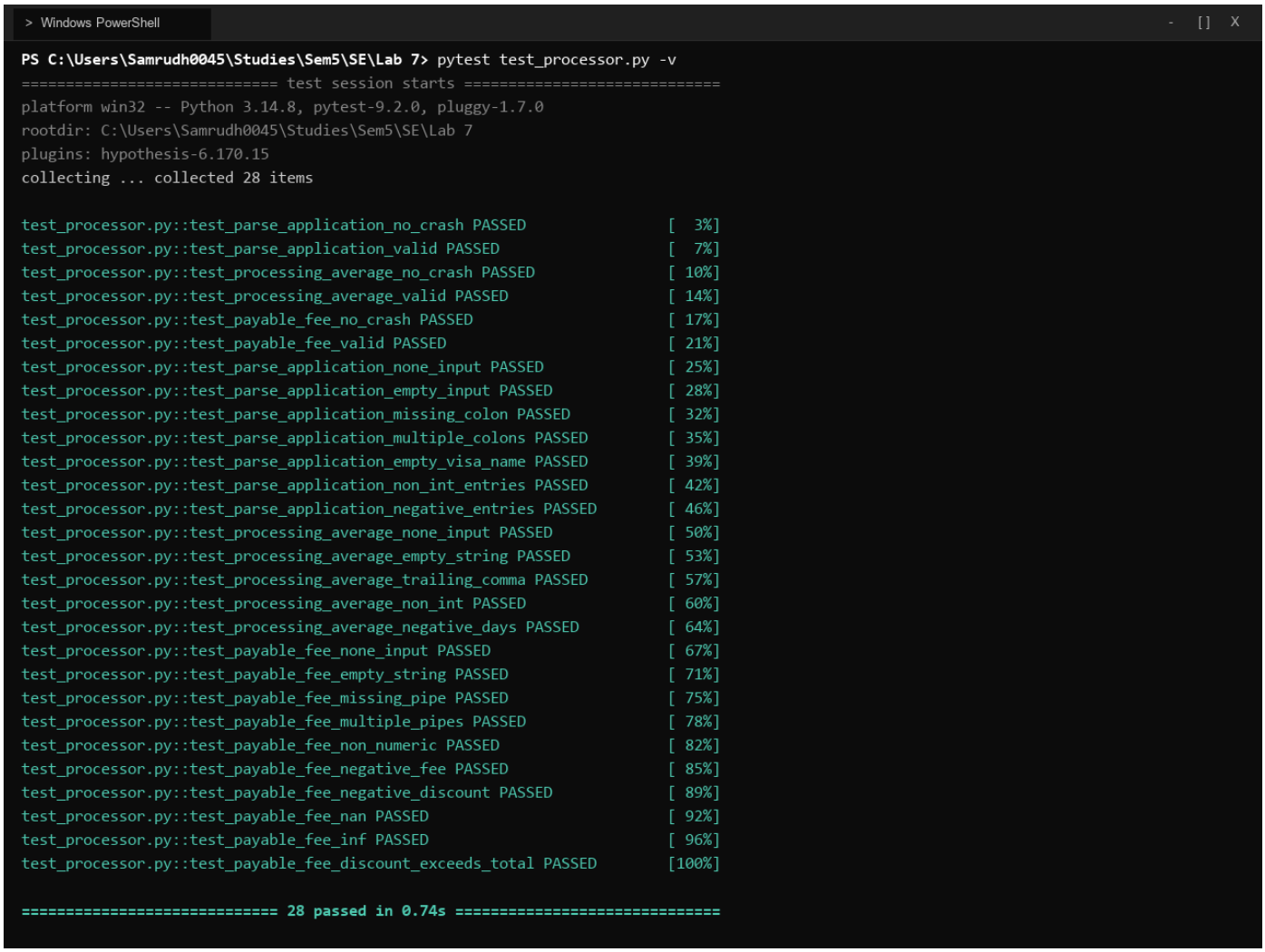


Figure 4: Final test execution showing all 28 tests passing cleanly with 100% success.

4. Bug Identification & Input Validation Summary

The table below summarizes all input validation bugs discovered during fuzz testing and the specific fixes implemented:

Function	Bug / Failing Input	Original Behavior	Implemented Fix
parse_application	text = None	Crashed with unhandled AttributeError on .split(':')	Added isinstance(text, str) check; raises ValueError.
parse_application	Missing or multiple colons (e.g. '', 'a:b:c')	Crashed with unpack ValueError (wrong number of elements)	Checked len(parts) == 2 before unpacking.
parse_application	Non-integer or negative entries (e.g. 'tourist:-3')	Negative entries were accepted silently without any validation	Converted with int() in try-except; enforced entries >= 0.
parse_application	Empty visa type (e.g. ' :2')	Returned an invalid empty visa tuple ('', 2)	Checked if not visa: raise ValueError().
processing_average	days_csv = None or empty ''	Crashed with AttributeError or ValueError on int('')	Validated string type and checked for empty tokens.
processing_average	Negative processing days (e.g. '5,-10,15')	Computed meaningless negative average days	Enforced day >= 0 for each CSV item.
processing_average	Trailing comma (e.g. '5,10,')	Crashed with ValueError on empty trailing item	Disallowed empty tokens: any(not x.strip() for x in parts).
payable_fee	text = None or missing pipe	Crashed with AttributeError or unpack failure	Enforced string type and exact 2-part split on '|'.
payable_fee	Negative fee or negative discount	Negative fee accepted; negative discount acted as fee increase	Enforced fee >= 0 and discount >= 0.
payable_fee	Non-finite numbers ('nan|500', 'inf|500')	Produced invalid NaN/Inf calculations	Added math.isfinite() validation check.

5. Reflection Questions & Key Insights

1. What makes fuzz testing different from standard unit testing?

Standard unit testing relies on hand-crafted test cases that generally check 'happy paths' or obvious boundaries that the developer already anticipated. In contrast, fuzz testing bombards the code with hundreds of randomly generated, semi-structured, and malformed inputs. This exposes unhandled edge cases that developers typically overlook, such as unexpected types, null values, empty strings, and negative numbers.

2. How does the Hypothesis shrinking mechanism assist in debugging?

When Hypothesis finds an input that triggers a crash, the initial input might be an unwieldy string with dozens of random characters. Hypothesis automatically 'shrinks' the failing case through iterative simplification to find the simplest possible minimal reproduction case (e.g., shrinking a long corrupt string down to None or an empty string ''). This makes debugging the root cause immediate and obvious.

3. Why should input validation raise specific, documented exceptions?

Allowing low-level runtime errors like AttributeError or ZeroDivisionError to bubble up causes unexpected application crashes and leaks internal implementation details. Enforcing input validation at the boundary and raising clean, consistent ValueErrors ensures the caller can catch and handle invalid user inputs predictably without crashing.

6. Python Source Code: processor.py (Corrected Application)

Complete source code of the application with comprehensive input validation:

```
"""
Lab 7: Fuzz Testing for Input Validation Bugs
Module: processor.py (Corrected Application)

This module provides functions for parsing visa applications,
calculating average processing days, and computing payable visa fees.
All functions include rigorous input validation to guard against
malformed inputs, unexpected data types, negative values, and edge cases.
"""

import math


def parse_application(text):
    """
    Parses a visa application string formatted as 'visa_type:entries'.

    Args:
        text (str): Input string containing visa type and number of entries.

    Returns:
        tuple[str, int]: Cleaned title-cased visa type and integer entry count.

    Raises:
        ValueError: If input is not a string, format is malformed, visa type is empty,
                    or entries count is invalid / negative.
    """
    if not isinstance(text, str):
        raise ValueError("Input must be a valid string")

    parts = text.split(':')
    if len(parts) != 2:
        raise ValueError("Invalid application format: expected 'visa_type:entries'")

    visa = parts[0].strip()
    entries_str = parts[1].strip()

    if not visa:
        raise ValueError("Visa type cannot be empty")

    try:
        entries = int(entries_str)
    except ValueError:
        raise ValueError("Entries must be a valid integer")

    if entries < 0:
        raise ValueError("Entries count cannot be negative")

    return visa.title(), entries


def processing_average(days_csv):
    """
    Calculates the integer average of processing days from a comma-separated string.

    Args:
        days_csv (str): Comma-separated list of processing days (e.g. '5,10,21').

    Returns:
        int: Floor average of the processing days.

    Raises:
        ValueError: If input is not a string, contains empty values, has non-integers,
                    or contains negative numbers.
    """
    if not isinstance(days_csv, str):
        raise ValueError("Input must be a valid string")

    parts = days_csv.split(',')
    if not parts or any(not item.strip() for item in parts):
        raise ValueError("Invalid processing days format: cannot contain empty values")

    try:
        values = [int(item.strip()) for item in parts]
    except ValueError:
        raise ValueError("Processing days must be integers")

    if any(day < 0 for day in values):
```

```
        raise ValueError("Processing days cannot be negative")

    if len(values) == 0:
        raise ValueError("No processing days provided")

    return sum(values) // len(values)


def payable_fee(text):
    """
    Calculates the net payable fee after applying 18% tax and subtracting any discount.
    Formula: max(fee * 1.18 - discount, 0)

    Args:
        text (str): Pipe-separated string formatted as 'fee|discount' (e.g. '6000|500').

    Returns:
        float: Payable fee, capped at minimum 0.

    Raises:
        ValueError: If input is not a string, format is malformed, values are non-numeric,
                    infinite/NaN, or negative.
    """
    if not isinstance(text, str):
        raise ValueError("Input must be a valid string")

    parts = text.split('|')
    if len(parts) != 2:
        raise ValueError("Invalid fee format: expected 'fee|discount'")

    try:
        fee = float(parts[0].strip())
        discount = float(parts[1].strip())
    except ValueError:
        raise ValueError("Fee and discount must be valid numbers")

    if not math.isfinite(fee) or not math.isfinite(discount):
        raise ValueError("Fee and discount must be finite numbers")

    if fee < 0 or discount < 0:
        raise ValueError("Fee and discount cannot be negative")

    total = fee * 1.18 - discount
    return max(total, 0.0)


def main():
    """Interactive command-line interface for the application."""
    print("=" * 45)
    print("    Visa Application Processing System")
    print("=" * 45)

    print("\n==== 1. parse_application ====")
    user_input = input("Enter visa type:entries (e.g. tourist:2): ")
    try:
        result = parse_application(user_input)
        print("Visa Type and Entries:", result)
    except ValueError as err:
        print(f"Error: {err}")

    print("\n==== 2. processing_average ====")
    user_input = input("Enter processing days as CSV (e.g. 5,10,21): ")
    try:
        result = processing_average(user_input)
        print("Average Processing Days:", result)
    except ValueError as err:
        print(f"Error: {err}")

    print("\n==== 3. payable_fee ====")
    user_input = input("Enter fee|discount (e.g. 6000|500): ")
    try:
        result = payable_fee(user_input)
        print("Payable Fee:", result)
    except ValueError as err:
        print(f"Error: {err}")


if __name__ == "__main__":
```

```
    main()
```

7. Python Source Code: test_processor.py (Hypothesis Test Suite)

Complete test suite containing property-based fuzz tests and edge-case validation tests:

```
"""
Lab 7: Fuzz Testing for Input Validation Bugs
Test Suite: test_processor.py

Uses Hypothesis to perform property-based and robustness fuzz testing
across parse_application, processing_average, and payable_fee.
"""

import math
import pytest
from hypothesis import given, strategies as st

from processor import (
    parse_application,
    processing_average,
    payable_fee
)


# ======================================================================
# 1. FUZZ TESTING FOR parse_application()
# ======================================================================

@given(st.one_of(st.text(), st.none()))
def test_parse_application_no_crash(s):
    """
    Robustness Fuzz Test:
    Feeds arbitrary text and None. The function must never crash with unhandled
    exceptions like AttributeError, TypeError, or IndexError.
    It must either return a valid (str, int) tuple or raise ValueError.
    """
    try:
        result = parse_application(s)
        assert isinstance(result, tuple)
        assert len(result) == 2
        assert isinstance(result[0], str)
        assert len(result[0]) > 0
        assert isinstance(result[1], int)
        assert result[1] >= 0
    except ValueError:
        pass


@given(
    st.text(
        alphabet=st.characters(
            blacklist_categories=("Cc", "Cs"),
            blacklist_characters=":"
        ),
        min_size=1
    ).filter(lambda x: bool(x.strip())),
    st.integers(min_value=0, max_value=100000)
)
def test_parse_application_valid(visa, entries):
    """
    Property-Based Fuzz Test:
    For any non-empty visa string without colons and any non-negative integer,
    the parsed output must match the stripped title-cased visa and the entry integer.
    """
    result = parse_application(f"{visa}:{entries}")
    assert result[0] == visa.strip().title()
    assert result[1] == entries


# ======================================================================
# 2. FUZZ TESTING FOR processing_average()
# ======================================================================

@given(st.one_of(st.text(), st.none()))
def test_processing_average_no_crash(s):
    """
    Robustness Fuzz Test:
    Feeds arbitrary text and None. The function must never crash with unhandled
    exceptions like AttributeError, ZeroDivisionError, or TypeError.
    It must either return a valid non-negative integer or raise ValueError.
    """
    try:
        result = processing_average(s)
```

```
        assert isinstance(result, int)
        assert result >= 0
    except ValueError:
        pass


@given(
    st.lists(
        st.integers(min_value=0, max_value=100000),
        min_size=1,
        max_size=100
    )
)
def test_processing_average_valid(values):
    """
    Property-Based Fuzz Test:
    For any non-empty list of non-negative integers formatted as CSV,
    the result must equal the mathematical integer floor average.
    """
    csv_str = ",".join(map(str, values))
    result = processing_average(csv_str)
    assert result == sum(values) // len(values)


# ============================================================================
# 3. FUZZ TESTING FOR payable_fee()
# ============================================================================

@given(st.one_of(st.text(), st.none()))
def test_payable_fee_no_crash(s):
    """
    Robustness Fuzz Test:
    Feeds arbitrary text and None. The function must never crash with unhandled
    exceptions like AttributeError, TypeError, or IndexError.
    It must either return a non-negative finite float/int or raise ValueError.
    """
    try:
        result = payable_fee(s)
        assert isinstance(result, (int, float))
        assert math.isfinite(result)
        assert result >= 0.0
    except ValueError:
        pass


@given(
    st.floats(min_value=0.0, max_value=1e6, allow_nan=False, allow_infinity=False),
    st.floats(min_value=0.0, max_value=1e6, allow_nan=False, allow_infinity=False)
)
def test_payable_fee_valid(fee, discount):
    """
    Property-Based Fuzz Test:
    For any valid non-negative finite fee and discount,
    payable_fee must compute max(fee * 1.18 - discount, 0).
    """
    result = payable_fee(f"{fee}|{discount}")
    expected = max(fee * 1.18 - discount, 0.0)
    assert result == pytest.approx(expected, rel=1e-5)


# ============================================================================
# 4. EDGE CASE & INPUT VALIDATION TESTS
# ============================================================================

def test_parse_application_none_input():
    with pytest.raises(ValueError, match="Input must be a valid string"):
        parse_application(None)


def test_parse_application_empty_input():
    with pytest.raises(ValueError, match="Invalid application format"):
        parse_application("")


def test_parse_application_missing_colon():
    with pytest.raises(ValueError, match="Invalid application format"):
        parse_application("tourist 2")
```

```
def test_parse_application_multiple_colons():
    with pytest.raises(ValueError, match="Invalid application format"):
        parse_application("tourist:2:express")


def test_parse_application_empty_visa_name():
    with pytest.raises(ValueError, match="Visa type cannot be empty"):
        parse_application("   :2")


def test_parse_application_non_int_entries():
    with pytest.raises(ValueError, match="Entries must be a valid integer"):
        parse_application("tourist:two")


def test_parse_application_negative_entries():
    with pytest.raises(ValueError, match="Entries count cannot be negative"):
        parse_application("tourist:-3")


def test_processing_average_none_input():
    with pytest.raises(ValueError, match="Input must be a valid string"):
        processing_average(None)


def test_processing_average_empty_string():
    with pytest.raises(ValueError, match="Invalid processing days format"):
        processing_average("")


def test_processing_average_trailing_comma():
    with pytest.raises(ValueError, match="Invalid processing days format"):
        processing_average("5,10,")


def test_processing_average_non_int():
    with pytest.raises(ValueError, match="Processing days must be integers"):
        processing_average("5,abc,10")


def test_processing_average_negative_days():
    with pytest.raises(ValueError, match="Processing days cannot be negative"):
        processing_average("5,-10,15")


def test_payable_fee_none_input():
    with pytest.raises(ValueError, match="Input must be a valid string"):
        payable_fee(None)


def test_payable_fee_empty_string():
    with pytest.raises(ValueError, match="Invalid fee format"):
        payable_fee("")


def test_payable_fee_missing_pipe():
    with pytest.raises(ValueError, match="Invalid fee format"):
        payable_fee("6000")


def test_payable_fee_multiple_pipes():
    with pytest.raises(ValueError, match="Invalid fee format"):
        payable_fee("6000|500|100")


def test_payable_fee_non_numeric():
    with pytest.raises(ValueError, match="Fee and discount must be valid numbers"):
        payable_fee("abc|500")


def test_payable_fee_negative_fee():
    with pytest.raises(ValueError, match="Fee and discount cannot be negative"):
        payable_fee("-6000|500")


def test_payable_fee_negative_discount():
    with pytest.raises(ValueError, match="Fee and discount cannot be negative"):
```

```
        payable_fee("6000|-500")


def test_payable_fee_nan():
    with pytest.raises(ValueError, match="Fee and discount must be finite numbers"):
        payable_fee("nan|500")


def test_payable_fee_inf():
    with pytest.raises(ValueError, match="Fee and discount must be finite numbers"):
        payable_fee("inf|500")


def test_payable_fee_discount_exceeds_total():
    # Fee = 100 -> total = 118 - 500 = -382 -> max(-382, 0) = 0.0
    assert payable_fee("100|500") == 0.0
```